

Code Arduino Motoréducteur avec encodeur

Le câblage électrique

Couleur Typique	Fonction	Connexion Arduino / Driver
Fils Moteur (gros)	Alimentation Moteur (+)	Sortie du Driver (BTS7960 / VNH2SP30)
Fils Moteur (gros)	Alimentation Moteur (-)	Sortie du Driver (BTS7960 / VNH2SP30)
Noir / Marron	Masse Encodeur (GND)	GND de l'Arduino
Rouge / Bleu	Alimentation Encodeur (VCC)	5V de l'Arduino
Jaune	Signal Encodeur Phase A	Pin 2 (Interruption 0)
Blanc	Signal Encodeur Phase B	Pin 4 (Entrée standard)

Pour le Moteur Droit : Phase A sur la **Pin 3** et Phase B sur la **Pin 5**

Le Code Arduino (Lecture + Calcul de Vitesse)

Ce code configure les interruptions pour compter les impulsions des deux moteurs en temps réel et calcule leur vitesse de rotation (RPM) toutes les 100 millisecondes. C'est la base avant d'attaquer le code de commande moteur (le PID).

```
// --- Configuration des Pins ---
```

```
const int ENCODER_L_A = 2; // Phase A Moteur Gauche (Impératif sur Pin 2)
```

```
const int ENCODER_L_B = 4; // Phase B Moteur Gauche
```

```
const int ENCODER_R_A = 3; // Phase A Moteur Droit (Impératif sur Pin 3)
```

```

const int ENCODER_R_B = 5; // Phase B Moteur Droit

// --- Variables de comptage (volatile car modifiées dans les interruptions) ---
volatile long ticksMoteurGauche = 0;
volatile long ticksMoteurDroit = 0;

// --- Paramètres pour le calcul de vitesse ---
unsigned long precedentMillis = 0;
const long intervalleLecon = 100; // Calcul toutes les 100ms

// À AJUSTER : Nombre de ticks par tour d'arbre de sortie (Dépend de votre réduction)
// Formule : Ticks_par_tour_moteur * Ratio_reduction
const float TICKS_PER_REV = 1200.0;

void setup() {
    Serial.begin(115200); // Vitesse haute pour ne pas bloquer les interruptions

    // Configuration des broches de l'encodeur en entrée avec résistance de pull-up
    pinMode(ENCODER_L_A, INPUT_PULLUP);
    pinMode(ENCODER_L_B, INPUT_PULLUP);
    pinMode(ENCODER_R_A, INPUT_PULLUP);
    pinMode(ENCODER_R_B, INPUT_PULLUP);

    // Attachement des interruptions sur le front montant (RISING) de la Phase A
    attachInterrupt(digitalPinToInterrupt(ENCODER_L_A), isrMoteurGauche, RISING);
    attachInterrupt(digitalPinToInterrupt(ENCODER_R_A), isrMoteurDroit, RISING);
}

```

```

void loop() {

    unsigned long actuelMillis = millis();

    // Toutes les 100ms, on calcule et on affiche la vitesse

    if (actuelMillis - precedentMillis >= intervalleLecon) {

        precedentMillis = actuelMillis;

        // Section critique : on bloque brièvement les interruptions pour lire proprement
        // les variables "long" (qui font 4 octets, l'Arduino 8-bits peut s'emmêler les pinceaux
        sinon)

        noInterrupts();

        long copieTicksG = ticksMoteurGauche;

        long copieTicksD = ticksMoteurDroit;

        ticksMoteurGauche = 0; // Remise à zéro pour la prochaine mesure

        ticksMoteurDroit = 0;

        interrupts();

        // Calcul de la vitesse en RPM (Tours par minute)
        // Delta_Ticks / Ticks_par_tour * (60 secondes / 0.1 seconde)

        float rpmGauche = (copieTicksG / TICKS_PER_REV) * (60.0 / (intervalleLecon / 1000.0));

        float rpmDroit = (copieTicksD / TICKS_PER_REV) * (60.0 / (intervalleLecon / 1000.0));

        // Affichage dans le moniteur série / Traceur série

        Serial.print("RPM_Gauche:");

        Serial.print(rpmGauche);

        Serial.print("\tRPM_Droit:");

        Serial.println(rpmDroit);

    }
}

```

```
}
```

```
// --- Routine d'Interruption (ISR) Moteur Gauche ---
```

```
void isrMoteurGauche() {
```

```
    // Si la Phase A monte, on regarde l'état de la Phase B pour connaître le sens
```

```
    if (digitalRead(ENCODER_L_B) == LOW) {
```

```
        ticksMoteurGauche++; // Marche avant
```

```
    } else {
```

```
        ticksMoteurGauche--; // Marche arrière
```

```
    }
```

```
}
```

```
// --- Routine d'Interruption (ISR) Moteur Droit ---
```

```
void isrMoteurDroit() {
```

```
    if (digitalRead(ENCODER_R_B) == LOW) {
```

```
        ticksMoteurDroit++; // Marche avant
```

```
    } else {
```

```
        ticksMoteurDroit--; // Marche arrière
```

```
    }
```

```
}
```

Comprendre la formule de calcul

Le code utilise la formule physique de la vitesse de rotation suivante :

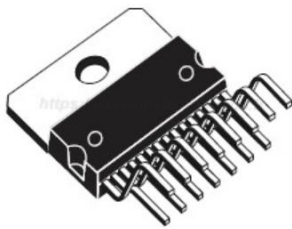
$$\text{RPM} = \frac{\Delta \text{Ticks}}{\text{PPR}} \times \frac{60}{\Delta t}$$

Où :

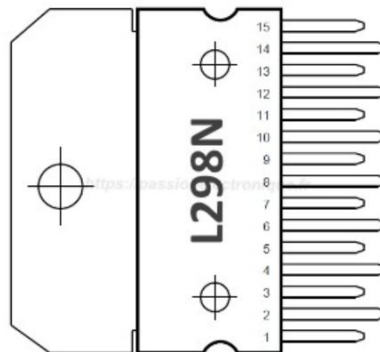
- ΔTicks est le nombre d'impulsions lues durant l'intervalle.
- PPR (Pulses Per Revolution) est le nombre total de ticks générés par l'encodeur pour un tour complet de la roue (valeur `TICKS_PER_REV` dans le code). *Note : Vérifiez bien la datasheet de votre modèle E20GM pour entrer la valeur exacte.*
- Δt est le temps écoulé en secondes (ici, 0.1 s).

L298N : brochage, et définition des pins

Vue en 3D



Vue de face



Broches

SensorB	: capteur de courant pont B
OUT4	: sortie 4 (pont B)
OUT3	: sortie 3 (pont B)
IN4	: entrée 4 (pont B)
ENB	: activation/désactivation pont B
IN3	: entrée 3 (pont B)
VSS	: ALIM partie "commande"
GND	: masse
IN2	: entrée 2 (pont A)
ENA	: activation/désactivation pont A
IN1	: entrée 1 (pont A)
VS	: ALIM partie "puissance"
OU2	: sortie 2 (pont A)
OUT1	: sortie 1 (pont A)
SensorA	: capteur de courant pont A



PassionElectronique.fr

Pont A = 1er pont en H (pouvant piloter un 1er moteur)
Pont B = 2nd pont en H (pouvant piloter un 2ème moteur)

Le Schéma de Câblage (Arduino ↔ L298N ↔ Moteurs)

Sur le schéma de votre fiche technique, repérez bien les pins de commande (IN1, IN2, ENA pour le Pont A / IN3, IN4, ENB pour le Pont B).

Composant L298N	Broche Arduino	Rôle
ENA (Pont A)	Pin 9 (PWM)	Vitesse Moteur Gauche
IN1	Pin 7	Direction Moteur Gauche (Avant)
IN2	Pin 8	Direction Moteur Gauche (Arrière)
ENB (Pont B)	Pin 10 (PWM)	Vitesse Moteur Droit
IN3	Pin 11	Direction Moteur Droit (Avant)
IN4	Pin 12	Direction Moteur Droit (Arrière)
GND	GND	Masse commune
VS	Borne + Batterie (11.1V)	Puissance pour les moteurs
VSS	5V Arduino	Alimentation de la logique du chip

Le Code Arduino Global (Moteurs + Encodeurs + PID)

Ce code complet fusionne la lecture de vos encodeurs E20GM, le contrôle de puissance via le L298N, et applique un double correcteur PID indépendant pour que chaque chenille atteigne précisément la vitesse demandée.

```
// --- PINS ENCODEURS ---
```

```
const int ENCODER_L_A = 2; // Interruption 0
```

```
const int ENCODER_L_B = 4;
```

```
const int ENCODER_R_A = 3; // Interruption 1
```

```
const int ENCODER_R_B = 5;
```

```
// --- PINS DRIVER L298N ---
```

```
const int ENA = 9; // Contrôle Vitesse Gauche (PWM)
```

```
const int IN1 = 7; // Sens Gauche
```

```
const int IN2 = 8; // Sens Gauche
```

```
const int ENB = 10; // Contrôle Vitesse Droit (PWM)
```

```
const int IN3 = 11; // Sens Droit
```

```
const int IN4 = 12; // Sens Droit
```

```
// --- VARIABLES ENCODEURS ---
```

```
volatile long ticksMoteurGauche = 0;
```

```
volatile long volatile ticksMoteurDroit = 0;
```

```
unsigned long precedentMillis = 0;
```

```
const long intervalle = 100; // Échantillonnage toutes les 100ms (dt = 0.1s)
```

```
const float TICKS_PER_REV = 750.0; // À ajuster selon votre test (ex: 3 PPR * 250)
```

```
// --- VARIABLES PID ---
```

```
float consigneG = 40.0; // Vitesse cible pour le moteur gauche (en RPM)
```

```
float consigneD = 40.0; // Vitesse cible pour le moteur droit (en RPM)
```

```
// Coefficients PID à affiner lors de vos tests à l'ESIEE
```

```
float Kp = 2.0;
```

```
float Ki = 0.5;
```

```
float Kd = 0.1;
```

```
float erreurPrecedenteG = 0, sommeErreursG = 0;
```

```
float erreurPrecedenteD = 0, sommeErreursD = 0;
```

```

void setup() {

  Serial.begin(115200);

  // Configuration L298N

  pinMode(ENA, OUTPUT); pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(ENB, OUTPUT); pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);

  // Configuration Encodeurs

  pinMode(ENCODER_L_A, INPUT_PULLUP); pinMode(ENCODER_L_B, INPUT_PULLUP);
  pinMode(ENCODER_R_A, INPUT_PULLUP); pinMode(ENCODER_R_B, INPUT_PULLUP);

  attachInterrupt(digitalPinToInterrupt(ENCODER_L_A), isrG, RISING);
  attachInterrupt(digitalPinToInterrupt(ENCODER_R_A), isrD, RISING);
}

void loop() {

  unsigned long actuelMillis = millis();

  if (actuelMillis - precedentMillis >= intervalle) {

    precedentMillis = actuelMillis;

    // 1. Lecture sécurisée des ticks

    noInterrupts();

    long copieG = ticksMoteurGauche; ticksMoteurGauche = 0;
    long copieD = ticksMoteurDroit; ticksMoteurDroit = 0;

    interrupts();

    // 2. Calcul des RPM réels

```



```

float rpmReelG = (copieG / TICKS_PER_REV) * (60.0 / (intervalle / 1000.0));
float rpmReelD = (copieD / TICKS_PER_REV) * (60.0 / (intervalle / 1000.0));

// 3. Calcul du PID - Moteur Gauche
float erreurG = consigneG - rpmReelG;
sommeErreursG += erreurG * 0.1;
float deriveeG = (erreurG - erreurPrecedenteG) / 0.1;
float commandeG = (Kp * erreurG) + (Ki * sommeErreursG) + (Kd * deriveeG);
erreurPrecedenteG = erreurG;

// 4. Calcul du PID - Moteur Droit
float erreurD = consigneD - rpmReelD;
sommeErreursD += erreurD * 0.1;
float deriveeD = (erreurD - erreurPrecedenteD) / 0.1;
float commandeD = (Kp * erreurD) + (Ki * sommeErreursD) + (Kd * deriveeD);
erreurPrecedenteD = erreurD;

// 5. Envoi des commandes de puissance au L298N
piloterMoteurGauche(commandeG);
piloterMoteurDroit(commandeD);

// 6. Debug / Affichage pour le traceur série
Serial.print("Target_G:"); Serial.print(consigneG);
Serial.print("\tRPM_G:");  Serial.print(rpmReelG);
Serial.print("\tTarget_D:"); Serial.print(consigneD);
Serial.print("\tRPM_D:");  Serial.println(rpmReelD);
}
}

```

```
// --- FONCTIONS DE PILOTAGE L298N ---  
  
void piloterMoteurGauche(float commande) {  
    int pwm = constrain((int)commande, -255, 255);  
    if (pwm >= 0) {  
        digitalWrite(IN1, HIGH);  
        digitalWrite(IN2, LOW);  
        analogWrite(ENA, pwm);  
    } else {  
        digitalWrite(IN1, LOW);  
        digitalWrite(IN2, HIGH);  
        analogWrite(ENA, abs(pwm));  
    }  
}
```

```
void piloterMoteurDroit(float commande) {  
    int pwm = constrain((int)commande, -255, 255);  
    if (pwm >= 0) {  
        digitalWrite(IN3, HIGH);  
        digitalWrite(IN4, LOW);  
        analogWrite(ENB, pwm);  
    } else {  
        digitalWrite(IN3, LOW);  
        digitalWrite(IN4, HIGH);  
        analogWrite(ENB, abs(pwm));  
    }  
}
```

```
// --- ROUTINES D'INTERRUPTIONS ---
```

```
void isrG() {
```

```
    if (digitalRead(ENCODER_L_B) == LOW) ticksMoteurGauche++; else ticksMoteurGauche--;  
}
```

```
void isrD() {
```

```
    if (digitalRead(ENCODER_R_B) == LOW) ticksMoteurDroit++; else ticksMoteurDroit--;  
}
```

Quelques règles de survie avec ce code sur le terrain :

1. **Le sens de rotation** : Si en demandant une consigne positive (consigneG = 40), le moteur tourne physiquement en arrière ou que la valeur RPM_G devient négative dans votre moniteur, inversez simplement les deux fils d'alimentation du moteur concerné connectés aux bornes bleues du L298N.
2. **Le Tuning à 6 cerveaux** : Mettez le robot sur une cale (les chenilles dans le vide) pour vos premiers tests. Modifiez les valeurs de Kp et Ki au début en laissant Kd = 0. Dès que les chenilles atteignent la consigne de manière stable sans s'arrêter ni osciller, posez l'Herbicator au sol et ajustez la puissance si le poids du robot nécessite de pousser les coefficients !